

MotionWorld D-011

Why an Isolated Unreal Bridge Component Is the Correct Experimental Boundary

Candidate study handout

30 August 2026 – Unreal Engine 5.8.2

Decision in one sentence. Keep Epic’s Game Animation Sample intact and add a small, source-controlled plugin component that supplies bounded desired-velocity commands to Mover and records finalized gameplay state after each simulation step.

1 Reading order and ownership standard

Read Sections 2–9 before approving D-011. Sections 10 and 11 are required before the first interview rehearsal.

Level	What you must be able to do without looking
MUST KNOW	Explain the boundary, action, authoritative state, tick order, residual equation, passthrough behavior, safety fields, and producer-ordering risk.
UNDERSTAND	Explain why the rejected alternatives are weaker and how each acceptance test could fail.
REFERENCE	Remember exact Unreal type/member spelling only well enough to locate it; do not memorize every API declaration.

2 The problem D-011 solves

The Game Animation Sample already provides a high-quality pawn, Mover/Smooth Walking dynamics, collision, and Motion Matching. MotionWorld needs only two new capabilities:

1. issue a programmatic planning action; and
2. observe the physical result at a well-defined simulation boundary.

Replacing the pawn would discard working animation integration. Editing the large Blueprint would create an opaque binary change that is hard to review and reproduce. The downloaded sample also contains a precompiled project module without visible project source. Composition is therefore the smallest reliable intervention: a plugin packages our code, and one actor component adds the bridge behavior to the existing pawn.

Plugin versus component

A **plugin** is a deployable source package: descriptor, C++ module, tests, and assets owned by MotionWorld. An **actor component** is the runtime object attached to the sample pawn. The plugin is the container; the component is the behavior.

3 Action semantics: request is not outcome

Let the planner action be desired planar world velocity

$$\mathbf{a}_t = \mathbf{v}_t^{\text{des}} = \begin{bmatrix} v_{x,t}^{\text{des}} & v_{y,t}^{\text{des}} \end{bmatrix}^T \quad [\text{cm/s}]. \quad (1)$$

Unreal receives a bounded three-dimensional command with zero vertical component:

$$\bar{\mathbf{a}}_t = \begin{cases} \mathbf{0}, & \mathbf{a}_t \text{ is non-finite,} \\ \mathbf{a}_t, & \|\mathbf{a}_t\|_2 \leq v_{\max}, \\ v_{\max} \frac{\mathbf{a}_t}{\|\mathbf{a}_t\|_2}, & \text{otherwise,} \end{cases} \quad \mathbf{u}_t = \begin{bmatrix} \bar{a}_{x,t} & \bar{a}_{y,t} & 0 \end{bmatrix}^T. \quad (2)$$

Radial clamping preserves direction. Independently clipping each axis can make a diagonal command faster than an axial command and can change its intended angle.

The bridge writes `EMoveInputType::Velocity` and $\mathbf{u}_t$ into `FCharacterDefaultInputs`. This does *not* set the executed velocity directly. Smooth Walking, collision, turning, the time step, and hidden spring state determine the result:

$$(\mathbf{s}_{t+1}, \mathbf{z}_{t+1}) = F_{\text{UE}}(\mathbf{s}_t, \mathbf{z}_t, \mathbf{u}_t, \boldsymbol{\xi}_t). \quad (3)$$

Here $\mathbf{z}_t$ is controller memory such as intermediate velocity and spring acceleration; $\boldsymbol{\xi}_t$ contains external effects such as collision or a push. The important interview sentence is: *the action is the request; the next state is Unreal's executed response*.

4 Input arbitration and passthrough

The existing Blueprint pawn already produces human input. UE 5.8.2 gathers the pawn input producer and compatible producer components at `BeginPlay`, then calls them before simulation. The proposed bridge obeys

$$\mathbf{u}_t^{\text{final}} = \begin{cases} \mathbf{u}_t^{\text{human}}, & m_t = 0, \\ \text{clamp}_{v_{\max}}(\mathbf{a}_t^{\text{planner}}), & m_t = 1, \end{cases} \quad (4)$$

where m_t is the automated-control flag. When $m_t = 0$, the bridge writes nothing. This **passthrough invariant** proves that installing the plugin does not silently change the baseline.

The installed implementation currently adds the pawn producer before gathered components. We expect the bridge to write later and win only in automated mode. This is a tested assumption, not a contract we hide. We inspect `GetLastInputCmd()` on every command test. If ordering is not stable, the fallback is an explicit composite input producer with deterministic arbitration.

5 Tick order and authoritative observation

The intended causal order for simulation step t is

$$\mathbf{o}_t \longrightarrow \mathbf{u}_t \xrightarrow{\text{ProduceInput}} F_{\text{UE}} \xrightarrow{\text{OnPostFinalize}} \mathbf{o}_{t+1}. \quad (5)$$

`OnPostMovement` is too early for logging because its state is explicitly mutable. `OnPostFinalize` runs after finalization on the game thread. The observation extracted from `FMoverDefaultSyncState` is

$$\mathbf{o}_t = [p_{x,t}, p_{y,t}, v_{x,t}, v_{y,t}, \cos \psi_t, \sin \psi_t, \omega_t]^T, \quad (6)$$

with position in cm, velocity in cm/s, yaw encoded without a discontinuity, and angular velocity in degrees/s unless deliberately converted. The Mover updated component/sync state is authoritative because it drives gameplay collision. The primary visual component is recorded separately for animation QA; it is not substituted for gameplay state.

Why encode facing as sine and cosine?

Raw yaw jumps from $+180^\circ$ to -180° although the physical orientation barely changes. The continuous pair $(\cos \psi, \sin \psi)$ avoids this artificial regression discontinuity. Recover yaw with

$$\psi = \text{atan2}(\sin \psi, \cos \psi). \quad (7)$$

6 Connection to the world model

The bridge creates aligned transition tuples

$$\mathcal{D} = \{(\mathbf{o}_t, \mathbf{u}_t, \mathbf{o}_{t+1})\}_{t=0}^{T-1}. \quad (8)$$

A nominal model predicts known movement structure:

$$\tilde{\mathbf{s}}_{t+1} = f_{\text{nom}}(\mathbf{s}_t, \mathbf{z}_t, \mathbf{u}_t). \quad (9)$$

The learning target is the systematic mismatch between Unreal and that fair baseline:

$$\mathbf{r}_t^* = \text{difference}(\mathbf{s}_{t+1}^{\text{UE}}, \tilde{\mathbf{s}}_{t+1}), \quad \hat{\mathbf{s}}_{t+1} = \text{compose}(\tilde{\mathbf{s}}_{t+1}, r_\theta(\mathbf{h}_t, \mathbf{u}_t)). \quad (10)$$

The bridge is therefore scientifically important, not plumbing. If command t is accidentally paired with state t rather than $t+1$, the model learns a timing artifact. If visual-root position is mixed with collision position, the target is internally inconsistent. If the command that Unreal actually consumed is not logged, the dataset can contain mislabeled actions.

7 Protocol safety and episode isolation

Every command carries (q, e, k) : protocol version, episode identifier, and sequence number. Accept command c only if

$$\text{accept}(c) = [q_c = q_{\text{supported}}] \wedge [e_c = e_{\text{current}}] \wedge [k_c > k_{\text{last}}] \wedge [\mathbf{a}_c \in \mathbb{R}^2]. \quad (11)$$

The Unreal side clamps again even if Python already clamps. This is defense in depth: delayed packets, stale planner state, and service restarts must not control a new episode. On timeout the safe command is zero desired velocity; Smooth Walking then decelerates rather than teleporting.

Reset must restore more than position:

$$\mathcal{R} = \{\text{transform, linear/angular velocity, Mover spring state, command, IDs, gate phase, history, planner warm start}\}. \quad (12)$$

Otherwise episode i can influence episode $i + 1$, creating hidden state leakage and invalid paired comparisons.

8 Alternatives and tradeoffs

Alternative	Why D-011 is preferable
Edit sample Blueprint	Binary diff is hard to review; graph is large; licensed asset becomes part of our implementation seam.
Replace sample pawn	Risks breaking Motion Matching, camera, collision, and movement-mode setup.
Edit sample module	Download contains its precompiled binary but no visible project source.
Use ordinary actor Tick	Ordering relative to Mover can be ambiguous; can log pre-move state or duplicate a simulation step.
Call MoverExamples helper	Sample pawn derives from <code>APawn</code> , not <code>AMoverExamplesCharacter</code> ; the helper is not its API.
Use <code>OnPostMovement</code>	State remains mutable; logged observation may not equal finalized gameplay state.

9 Acceptance tests

D-011 is accepted only after evidence for each invariant:

1. **Isolation:** empty plugin compiles without modifying Epic assets.
2. **Passthrough:** plugin disabled preserves human locomotion.
3. **Command identity:** fixed command equals `GetLastInputCmd()` after clamping.
4. **Dynamics:** zero, forward, lateral, diagonal, stop, and reverse produce plausible Smooth Walking responses rather than teleportation.
5. **Temporal alignment:** sequence and simulation time are monotonic; action t is paired with finalized state $t + 1$.
6. **Frame/unit contract:** every field declares world/local frame and unit.
7. **State separation:** gameplay and visual transforms have different field names.
8. **Reset isolation:** repeated same-seed episodes match within declared tolerance.

10 Memory warning: what it could mean

On Apple Silicon, CPU and GPU share unified physical memory, but Unreal can still report distinct budgets. The exact warning text is required before changing settings.

Warning family	Meaning	Evidence needed
macOS memory pressure	Total resident/allocated memory approaches system capacity.	Activity Monitor memory pressure, swap, process resident size.
Metal/video-memory budget	Render resources exceed an RHI-reported GPU budget.	Exact on-screen warning and Unreal log lines around it.
Texture streaming pool	Resident textures exceed the configured streaming pool.	Exact “texture pool over budget” message and <code>stat streaming</code> .
Build/derived-data estimate	Shader or texture build reports temporary workspace, not necessarily live exhaustion.	Wording, timestamp, and whether it disappears after initial processing.

Current evidence shows successful 60 Hz Play-in-Editor and no explicit OOM, video-memory exhaustion, or texture-pool-over-budget line in the project log. Therefore the warning is unresolved, not ignored. It matters because memory pressure can cause frame-time spikes and invalidate planner p95 latency even when movement remains functionally correct.

11 Interview defense and teach-back

You must answer these without reading:

1. Why is a plugin different from an actor component?
2. Why is desired velocity an action rather than the next velocity?
3. Why is `OnPostFinalize` preferable to actor Tick or `OnPostMovement`?
4. What makes Mover state authoritative, and why is the visual root separate?
5. What producer-ordering assumption could invalidate D-011, and how do we test it?
6. Why does passthrough mode matter to baseline fairness?
7. How could an off-by-one action/state pairing corrupt residual learning?
8. Why must reset include hidden controller state and episode identity?
9. If D-011 fails, what is the smallest fallback?

One-minute answer structure: existing system → smallest intervention → exact command boundary → exact observation boundary → safety/fairness invariant → runtime test that could falsify the design.